

Script-Controlled ACL – Restrict Record Access Based on Field Value

1. Project Description

Script-Controlled ACL – Restrict Record Access Based on Field Value is a ServiceNow security project that demonstrates how **Access Control Lists (ACLs)** can be used to control access to records based on user roles and field values.

The project implements separate ACL rules for **READ, CREATE, WRITE, and DELETE** operations. Script conditions are used in the ACLs to check the value of a particular field and determine whether the user should be allowed or denied access.

The main objective of this project is to provide **field-level and record-level security** in a ServiceNow application.

2. Scenarios

Scenario 1: Authorized User Access

A user with the required role attempts to access a record. The ACL checks the user's role and the field value before allowing access.

Scenario 2: Unauthorized User Access

A user without the required role attempts to access a restricted record. The ACL denies the requested operation.

Scenario 3: Field-Based Record Restriction

Access to a record is controlled based on the value stored in a specific field.

For example, if the field value is "**Restricted**", the ACL script can allow or deny access depending on the configured security rule.

Scenario 4: Operation-Based Access Control

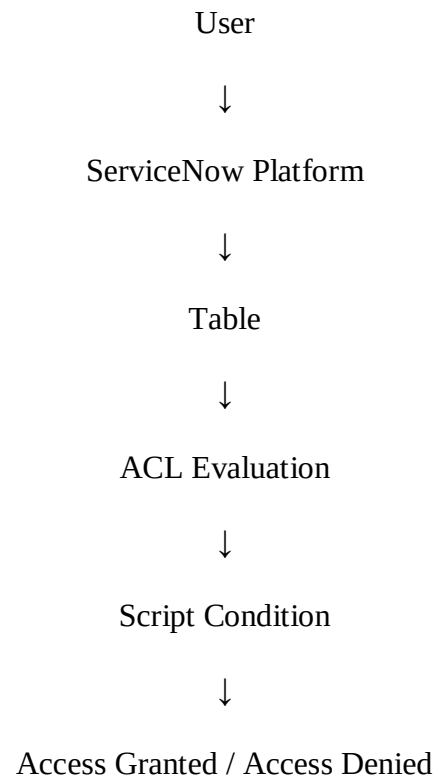
Different ACLs are created for different operations:

Operation	Purpose
READ	Controls viewing of records
CREATE	Controls creation of new records

WRITE	Controls modification of records
DELETE	Controls deletion of records

3. Technical Architecture

3.1 Architecture Diagram



3.2 Main Components

Component	Description
User	Person accessing the ServiceNow application
Role	Determines the permissions assigned to the user
Table	Stores application records
Field	Contains the value used for access validation
ACL	Defines access permissions for a particular operation
ACL Script	Contains JavaScript logic used to evaluate access
ServiceNow Security Engine	Evaluates ACL rules before allowing access

3.3 Data Flow

1. User requests access to a record.
2. ServiceNow checks the applicable ACL.
3. The user's role is verified.
4. The ACL script is executed.
5. The required field value is checked.
6. Access is either **Granted** or **Denied**.

3.4 ACL Operations

ACL Type	Function
READ	Controls whether a user can view a record
CREATE	Controls whether a user can create a record
WRITE	Controls whether a user can modify a record
DELETE	Controls whether a user can delete a record

Database Note

The project uses the **ServiceNow platform database**. Therefore, a traditional database ER diagram is not required.

Instead, the project can represent the database/security design using:

- Table Structure
- Field Structure
- ACL Configuration
- ACL Security Flow Diagram

4. Pre-Requisites

The following are required for developing and testing the project:

1. ServiceNow Developer Instance
2. Basic knowledge of ServiceNow
3. Knowledge of Users and Roles
4. Knowledge of Tables and Fields
5. Understanding of Access Control Lists
6. Basic JavaScript knowledge
7. Knowledge of ACL Conditions and Scripts
8. Understanding of CRUD operations
9. Knowledge of `GlideRecord`

10. Ability to use user impersonation for testing

5. Project Workflow

The project is divided into seven milestones.

Milestone	Activity	Completion
Milestone 1	Creation of Users and Roles	90%
Milestone 2	Tables Creation	90%
Milestone 3	Creation of Access Control List – READ	90%
Milestone 4	Creation of Access Control List – CREATE	90%
Milestone 5	Creation of Access Control List – WRITE	90%
Milestone 6	Creation of Access Control List – DELETE	90%
Milestone 7	Conclusion	90%

6. Detailed Milestones and Activities

Milestone 1: Creation of Users and Roles

Activity 1.1: Create Users

Create the required users in the ServiceNow instance.

Navigate to:

User Administration → Users

Create the required user accounts with appropriate information.

Activity 1.2: Create Roles

Create the roles required for the project.

Navigate to:

User Administration → Roles

Roles are used to identify which users should have permission to perform specific operations.

Activity 1.3: Assign Roles to Users

Assign the required roles to the created users.

Navigate to the user record and add the required role.

Activity 1.4: Verify User Access

Use **Impersonate User** to verify the permissions of different users.

This helps test whether the ACL rules work correctly for authorized and unauthorized users.

Milestone 2: Tables Creation

Activity 2.1: Create Application Table

Create a custom table to store the records used in the project.

Navigate to:

System Definition → Tables

Create the required custom table.

Activity 2.2: Create Fields

Add the required fields to the table.

Example fields may include:

Field	Purpose
Name	Stores the record name
Description	Stores record information
Access Type	Determines the access level
Created By	Stores the creator information

Activity 2.3: Configure Field Values

Configure the field that will be used by the ACL script.

Example values:

- Restricted
- Normal
- Public

Activity 2.4: Create Sample Records

Create sample records to test the ACL rules.

Record Access Type

Record 1 Restricted

Record 2 Normal

Record 3 Public

Activity 2.5: Verify Table Data

Open the table and verify that the records and field values have been created correctly.

Milestone 3: Creation of Access Control List – READ

Activity 3.1: Create READ ACL

Create an ACL for the **READ** operation.

The READ ACL controls whether a user can view a particular record.

Activity 3.2: Configure Table and Operation

Configure the ACL with:

Configuration	Value
Table	Custom ServiceNow Table
Operation	READ
Role	Required User Role
Condition	Field-based condition

Activity 3.3: Add Script Condition

The ACL script checks the value of the selected field.

Example:

```
answer = false;

if (current.access_type == "Restricted") {
    answer = true;
}
```

Note: The field name and value should be changed according to the actual table configuration.

Activity 3.4: Test READ Access

Test the ACL using users with different roles and records with different field values.

Activity 3.5: Verify Result

Verify that authorized users can read the permitted records while unauthorized users are denied access.

Milestone 4: Creation of Access Control List – CREATE

Activity 4.1: Create CREATE ACL

Create an ACL for the **CREATE** operation.

The CREATE ACL controls whether a user can create a new record.

Activity 4.2: Configure CREATE ACL

Configure the table, operation, role, and required conditions.

Configuration	Value
Table	Custom ServiceNow Table
Operation	CREATE
Role	Required User Role
Condition	Field-based condition

Activity 4.3: Add Script Logic

Add a script to validate the required field value before allowing record creation.

Activity 4.4: Test Record Creation

Attempt to create records using authorized and unauthorized users.

Activity 4.5: Verify Result

Verify that only users satisfying the ACL conditions can create records.

Milestone 5: Creation of Access Control List – WRITE

Activity 5.1: Create WRITE ACL

Create an ACL for the **WRITE** operation.

The WRITE ACL controls whether a user can modify an existing record.

Activity 5.2: Configure WRITE ACL

Configuration	Value
Table	Custom ServiceNow Table
Operation	WRITE
Role	Required User Role
Condition	Field-based condition

Activity 5.3: Add Script Logic

The script checks the required field value before allowing the user to modify the record.

Activity 5.4: Test Record Modification

Try modifying records using different users and field values.

Activity 5.5: Verify Result

Verify that authorized users can modify permitted records while unauthorized users cannot modify restricted records.

Milestone 6: Creation of Access Control List – DELETE

Activity 6.1: Create DELETE ACL

Create an ACL for the **DELETE** operation.

The DELETE ACL controls whether a user can delete a record.

Activity 6.2: Configure DELETE ACL

Configuration	Value
Table	Custom ServiceNow Table
Operation	DELETE
Role	Required User Role
Condition	Field-based condition

Activity 6.3: Add Script Logic

Add the required script condition to determine whether the user is allowed to delete the record.

Activity 6.4: Test Record Deletion

Attempt to delete records using authorized and unauthorized users.

Activity 6.5: Verify Result

Verify that the DELETE operation is allowed only when the ACL conditions are satisfied.

Milestone 7: Conclusion

Activity 7.1: Verify All ACL Operations

Verify all four ACL operations:

- READ
- CREATE
- WRITE
- DELETE

Activity 7.2: Test Different Users

Use different users and roles to verify the access restrictions.

Activity 7.3: Verify Field-Based Security

Verify that the ACL script correctly evaluates the selected field value before granting or denying access.

Activity 7.4: Final Result

The project demonstrates how ServiceNow **Script-Controlled ACLs** can be used to implement secure, field-based access control.

7. Example Configuration

Configuration	Example
Table	Custom ServiceNow Table
Field	Access Type
Example Value	Restricted
ACL Operations	READ, CREATE, WRITE, DELETE
Security Method	Role + Script Condition

8. Exploring ServiceNow Application

8.1 Users and Roles Page

This section is used to create users, create roles, and assign roles to users.

8.2 Tables and Records Page

This section is used to create the application table, fields, and sample records.

8.3 READ ACL Configuration

The READ ACL controls whether the user can view the selected records.

8.4 CREATE ACL Configuration

The CREATE ACL controls whether the user can create new records.

8.5 WRITE ACL Configuration

The WRITE ACL controls whether the user can modify existing records.

8.6 DELETE ACL Configuration

The DELETE ACL controls whether the user can delete records.

8.7 Access Granted Result

When the user's role and field value satisfy the ACL conditions, ServiceNow allows the requested operation.

8.8 Access Denied Result

When the ACL conditions are not satisfied, ServiceNow prevents the user from performing the requested operation.

9. Testing

Testing is performed to verify that each ACL works according to the defined security rules.

9.1 READ ACL Testing

User	Field Value	Expected Result
Authorized User	Restricted	Access Allowed
Unauthorized User	Restricted	Access Denied

9.2 CREATE ACL Testing

User	Condition	Expected Result
Authorized User	Valid	Record Created
Unauthorized User	Invalid	Access Denied

9.3 WRITE ACL Testing

User	Field Value	Expected Result
Authorized User	Allowed Value	Record Updated

Unauthorized User	Restricted Value	Access Denied
-------------------	------------------	---------------

9.4 DELETE ACL Testing

User	Field Value	Expected Result
Authorized User	Allowed Value	Record Deleted
Unauthorized User	Restricted Value	Access Denied

10. Conclusion

The **Script-Controlled ACL – Restrict Record Access Based on Field Value** project demonstrates the implementation of security controls in the ServiceNow platform.

The project uses **Users, Roles, Tables, Fields, ACLs, and JavaScript scripts** to control access to records.

Separate ACLs are created for **READ, CREATE, WRITE, and DELETE** operations. The ACL scripts evaluate field values and user permissions before allowing an operation.

This project provides a practical understanding of how **role-based and field-based access control** can be implemented in ServiceNow.

Future Enhancements

The project can be enhanced by adding:

- More user roles and permission levels
- Multiple field-based security conditions
- Dynamic access rules
- Department-based access control
- Group-based permissions
- Advanced scripted ACL conditions
- Security auditing and logging
- More complex record-level restrictions